

Big Data Analytics Lab — All Programs

TE/V (A & B) • Academic Year 2026–27 • CEPEL5014

Complete practical-exam programs and commands — Ubuntu/Linux. Includes MongoDB and HBase for Experiment 3.

EXPERIMENT 1 — HADOOP HDFS PRACTICAL

HDFS basics, Hadoop checking/starting, copying, retrieving, moving, displaying and deleting files.

Check Java

```
java -version
```

Check Hadoop

```
hadoop version
```

Start Hadoop and check processes

```
start-dfs.sh  
start-yarn.sh  
jps
```

Create HDFS directories

```
hdfs dfs -mkdir /bda  
hdfs dfs -mkdir /bda/input
```

List directories

```
hdfs dfs -ls /  
hdfs dfs -ls /bda
```

Create local input file

```
echo "hello hadoop big data" > data.txt  
cat data.txt
```

Copy local file to HDFS

```
hdfs dfs -put data.txt /bda/input/  
hdfs dfs -ls /bda/input
```

Display HDFS file

```
hdfs dfs -cat /bda/input/data.txt
```

Copy HDFS file to local

```
mkdir output  
hdfs dfs -get /bda/input/data.txt output/  
ls output
```

Alternative copy commands

```
hdfs dfs -copyFromLocal data.txt /bda/input/  
hdfs dfs -copyToLocal /bda/input/data.txt output/
```

Move, copy and delete

```
hdfs dfs -mv /bda/input/data.txt /bda/  
hdfs dfs -ls /bda  
hdfs dfs -cp /bda/data.txt /bda/input/  
hdfs dfs -rm /bda/data.txt  
hdfs dfs -rm -r /bda/input
```

Complete HDFS sequence

```
hdfs dfs -mkdir /bda  
hdfs dfs -mkdir /bda/input  
echo "hello hadoop big data" > data.txt  
hdfs dfs -put data.txt /bda/input/  
hdfs dfs -ls /bda/input  
hdfs dfs -cat /bda/input/data.txt  
mkdir output  
hdfs dfs -get /bda/input/data.txt output/  
hdfs dfs -mv /bda/input/data.txt /bda/  
hdfs dfs -ls /bda  
hdfs dfs -rm /bda/data.txt  
hdfs dfs -rm -r /bda/input
```

EXPERIMENT 2 — HADOOP MAPREDUCE: WORD COUNT

Complete Java MapReduce Word Count program.

WordCount.java

```
import java.io.IOException;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

    public static class TokenizerMapper
        extends Mapper<Object, Text, Text, IntWritable> {

        private final static IntWritable one =
            new IntWritable(1);

        private Text word = new Text();

        public void map(Object key, Text value, Context context)
            throws IOException, InterruptedException {

            String[] words =
                value.toString().split("\\s+");

            for (String w : words) {
                word.set(w);
                context.write(word, one);
            }
        }
    }

    public static class IntSumReducer
        extends Reducer<Text, IntWritable,
            Text, IntWritable> {

        public void reduce(Text key,
            Iterable<IntWritable> values,
            Context context)
            throws IOException, InterruptedException {

            int sum = 0;

            for (IntWritable value : values) {
                sum += value.get();
            }

            context.write(
                key,
                new IntWritable(sum)
            );
        }
    }

    public static void main(String[] args)
        throws Exception {

        Configuration conf =
            new Configuration();

        Job job =
            Job.getInstance(conf, "word count");

        job.setJarByClass(WordCount.class);

        job.setMapperClass(
            TokenizerMapper.class);

        job.setReducerClass(
            IntSumReducer.class);

        job.setOutputKeyClass(
            Text.class);
```

```

        job.setOutputValueClass(
            IntWritable.class);

        FileInputFormat.addInputPath(
            job,
            new Path(args[0]));

        FileOutputFormat.setOutputPath(
            job,
            new Path(args[1]));

        System.exit(
            job.waitForCompletion(true)
                ? 0 : 1
        );
    }
}

```

Input file — input.txt

```

hello world
hello hadoop
hadoop world

```

Compile and create JAR

```

javac -classpath "$(hadoop classpath)" -d . WordCount.java
jar -cvf wordcount.jar *.class

```

Upload input and run

```

hdfs dfs -mkdir -p /wordcount/input
hdfs dfs -put input.txt /wordcount/input/
hdfs dfs -cat /wordcount/input/input.txt

hdfs dfs -rm -r /wordcount/output
hadoop jar wordcount.jar WordCount /wordcount/input /wordcount/output

hdfs dfs -ls /wordcount/output
hdfs dfs -cat /wordcount/output/part-r-00000

```

Expected output

```

hadoop 2
hello 2
world 2

```

MapReduce flow

```

Input
↓
Mapper
↓
(word, 1)
↓
Shuffle & Sort
↓
Reducer
↓
(word, count)

```

EXPERIMENT 3 — MONGODB / HBASE NoSQL

This experiment requires installation/configuration of MongoDB or HBase and execution of NoSQL commands. Both command sets are included here.

PART A — MongoDB

Start MongoDB on Ubuntu

```
sudo systemctl start mongod
sudo systemctl status mongod
mongosh
```

Database and collection

```
use bda
db.createCollection("students")
show collections
```

Insert one document

```
db.students.insertOne({
  id: 1,
  name: "Amit",
  age: 21,
  branch: "Computer",
  marks: 78
})
```

Insert multiple documents

```
db.students.insertMany([
  { id: 2, name: "Priya", age: 20, branch: "Computer", marks: 92 },
  { id: 3, name: "Rahul", age: 22, branch: "IT", marks: 67 },
  { id: 4, name: "Neha", age: 21, branch: "Computer", marks: 85 },
  { id: 5, name: "Riya", age: 20, branch: "IT", marks: 95 }
])
```

Read / query

```
db.students.find()
db.students.find().pretty()
db.students.find({branch: "Computer"})
db.students.find({marks: {$gt: 80}})
db.students.find({marks: {$lt: 80}})
db.students.find({marks: {$gte: 70, $lte: 90}})
```

Update

```
db.students.updateOne(
  {name: "Amit"},
  {$set: {marks: 88}}
)
db.students.find({name: "Amit"})
```

Delete, count and sort

```
db.students.deleteOne({name: "Rahul"})
db.students.countDocuments()
db.students.find().sort({marks: 1})
db.students.find().sort({marks: -1})
```

Drop collection

```
db.students.drop()
```

MongoDB CRUD shortcut

```
CREATE → insertOne() / insertMany()
READ → find()
UPDATE → updateOne()
DELETE → deleteOne()
```

PART B — HBase

Start HBase on Ubuntu

```
start-hbase.sh
jps
```

Open HBase shell

```
hbase shell
```

Create table and list tables

```
create 'students', 'info'
```

```
list
```

Describe table

```
describe 'students'
```

Insert first student

```
put 'students', '1', 'info:name', 'Amit'  
put 'students', '1', 'info:age', '21'  
put 'students', '1', 'info:branch', 'Computer'  
put 'students', '1', 'info:marks', '78'
```

Insert second student

```
put 'students', '2', 'info:name', 'Priya'  
put 'students', '2', 'info:age', '20'  
put 'students', '2', 'info:branch', 'Computer'  
put 'students', '2', 'info:marks', '92'
```

Display all records

```
scan 'students'
```

Get one row

```
get 'students', '1'
```

Get a specific column

```
get 'students', '1', 'info:name'
```

Update data

```
put 'students', '1', 'info:marks', '88'  
get 'students', '1'
```

Delete a specific cell

```
delete 'students', '1', 'info:marks'  
get 'students', '1'
```

Delete an entire row

```
deleteall 'students', '2'  
scan 'students'
```

Count rows

```
count 'students'
```

Disable and drop table

```
disable 'students'  
drop 'students'
```

Complete HBase practice sequence

```
create 'students', 'info'  
  
put 'students', '1', 'info:name', 'Amit'  
put 'students', '1', 'info:age', '21'  
put 'students', '1', 'info:branch', 'Computer'  
put 'students', '1', 'info:marks', '78'  
  
put 'students', '2', 'info:name', 'Priya'  
put 'students', '2', 'info:age', '20'  
put 'students', '2', 'info:branch', 'Computer'  
put 'students', '2', 'info:marks', '92'  
  
scan 'students'  
get 'students', '1'  
  
put 'students', '1', 'info:marks', '88'  
get 'students', '1'  
  
delete 'students', '1', 'info:marks'  
deleteall 'students', '2'  
  
count 'students'  
  
disable 'students'  
drop 'students'
```

EXPERIMENT 4 — HIVE DATABASE + DESCRIPTIVE ANALYTICS

Start Hive

```
hive
# or, where applicable:
beeline
```

Create/select database

```
CREATE DATABASE bda;
SHOW DATABASES;
USE bda;
```

Create table

```
CREATE TABLE students (
    id INT,
    name STRING,
    age INT,
    marks FLOAT
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ',';
```

Insert records

```
INSERT INTO students VALUES
(1, 'Amit', 21, 78),
(2, 'Priya', 20, 92),
(3, 'Rahul', 22, 67),
(4, 'Neha', 21, 85),
(5, 'Riya', 20, 95);
```

Display data

```
SELECT * FROM students;
```

Basic statistics

```
SELECT COUNT(*) FROM students;
SELECT AVG(marks) FROM students;
SELECT MIN(marks) FROM students;
SELECT MAX(marks) FROM students;
SELECT SUM(marks) FROM students;
SELECT STDDEV(marks) FROM students;
SELECT VARIANCE(marks) FROM students;
```

Complete descriptive statistics query

```
SELECT
    COUNT(marks) AS count,
    AVG(marks) AS average,
    MIN(marks) AS minimum,
    MAX(marks) AS maximum,
    SUM(marks) AS total,
    STDDEV(marks) AS standard_deviation,
    VARIANCE(marks) AS variance
FROM students;
```

EXPERIMENT 5 — DATA STREAM ALGORITHM: BLOOM FILTER

The experiment allows any one of DGIM, Bloom Filter, or Flajolet-Martin. Bloom Filter is selected here.

File: bloom_filter.py

```
import hashlib

class BloomFilter:

    def __init__(self, size):
        self.size = size
        self.bit_array = [0] * size

    def hash1(self, item):
        hash_value = hashlib.md5(
            item.encode()
        ).hexdigest()
        return int(hash_value, 16) % self.size

    def hash2(self, item):
        hash_value = hashlib.sha1(
            item.encode()
        ).hexdigest()
        return int(hash_value, 16) % self.size

    def add(self, item):
        index1 = self.hash1(item)
        index2 = self.hash2(item)

        self.bit_array[index1] = 1
        self.bit_array[index2] = 1

    def check(self, item):
        index1 = self.hash1(item)
        index2 = self.hash2(item)

        if (self.bit_array[index1] == 1 and
            self.bit_array[index2] == 1):
            return True

        return False

# Create Bloom Filter
bf = BloomFilter(10)

# Add elements
bf.add("apple")
bf.add("banana")
bf.add("mango")

# Check elements
print("apple:", bf.check("apple"))
print("banana:", bf.check("banana"))
print("mango:", bf.check("mango"))
print("orange:", bf.check("orange"))
```

Run

```
python3 bloom_filter.py
```

Expected output

```
apple: True
banana: True
mango: True
orange: False
```

Interpretation

```
Definitely NOT present
OR
Maybe present
```

EXPERIMENT 6 — DATA VISUALIZATION USING R

The experiment permits Hive/PIG/R/Tableau. R is selected here. These programs can be run directly in RStudio.

Program 1 — Bar Chart: visualization.R

```
# Student names
students <- c(
  "Amit",
  "Priya",
  "Rahul",
  "Neha",
  "Riya"
)

# Student marks
marks <- c(
  78,
  92,
  67,
  85,
  95
)

# Create bar chart
barplot(
  marks,
  names.arg = students,
  main = "Student Marks",
  xlab = "Students",
  ylab = "Marks"
)
```

Run from Ubuntu terminal

```
Rscript visualization.R
```

Program 2 — Line Chart

```
# Student names
students <- c(
  "Amit",
  "Priya",
  "Rahul",
  "Neha",
  "Riya"
)

# Marks
marks <- c(
  78,
  92,
  67,
  85,
  95
)

# Create line chart
plot(
  marks,
  type = "o",
  xaxt = "n",
  main = "Student Marks",
  xlab = "Students",
  ylab = "Marks"
)

axis(
  1,
  at = 1:length(students),
  labels = students
)
```

Program 3 — Pie Chart

```
# Student names
students <- c(
  "Amit",
  "Priya",
  "Rahul",
  "Neha",
  "Riya"
)

# Marks
marks <- c(
```



```
78,  
92,  
67,  
85,  
95  
)  
  
# Create pie chart  
pie(  
  marks,  
  labels = students,  
  main = "Student Marks Distribution"  
)
```

RStudio

Open RStudio → File → New File → R Script → paste the program → select/run the code with Ctrl+Enter. The graph appears in the Plots panel.

LAST-MINUTE REVISION

HDFS

```
hdfs dfs -mkdir
hdfs dfs -ls
hdfs dfs -put
hdfs dfs -get
hdfs dfs -copyFromLocal
hdfs dfs -copyToLocal
hdfs dfs -cat
hdfs dfs -mv
hdfs dfs -cp
hdfs dfs -rm
hdfs dfs -rm -r
```

MapReduce

```
Mapper
Reducer
Driver
Input → Mapper → (word,1) → Shuffle & Sort → Reducer → (word,count)
```

MongoDB

```
use bda
db.createCollection()
db.students.insertOne()
db.students.insertMany()
db.students.find()
db.students.updateOne()
db.students.deleteOne()
db.students.countDocuments()
db.students.drop()
```

HBase

```
create
list
describe
put
scan
get
delete
deleteall
count
disable
drop
```

Hive

```
CREATE DATABASE
SHOW DATABASES
USE
CREATE TABLE
INSERT
SELECT
COUNT
AVG
MIN
MAX
SUM
STDDEV
VARIANCE
```

Bloom Filter

```
hash1()
hash2()
add()
check()
bit array
false positive
```

R

```
barplot()
plot()
pie()
```

Ubuntu quick commands

```
java -version
hadoop version
start-dfs.sh
start-yarn.sh
jps
```

```
hbase shell  
python3 bloom_filter.py  
Rscript visualization.R  
xdg-open filename.pdf
```